Anomaly-Based Network Intrusion Detection Using Kolmogorov-Arnold Networks and Temporal Deep Learning: A Chronological NetFlow Benchmark

Vedant Kapur
School of Computer Science Engineering and Technology (SCSET)
Bennett University
Greater Noida, India
kapoorvedant18@gmail.com

Tanmay Kaul
School of Computer Science Engineering and Technology (SCSET)
Bennett University
Greater Noida, India
tanmaykauloff@gmail.com

Sanyam Jain
School of Computer Science Engineering and Technology (SCSET)
Bennett University
Greater Noida, India
sanyam13072006@gmail.com

***Abstract*—This paper compares Kolmogorov-Arnold networks (KANs) with recurrent and Transformer models for network intrusion detection using chronologically ordered NetFlow v3 data. The benchmark covers five dataset entries—four University of Queensland NetFlow v3 datasets and the composite BigFlow-NIDS release—and 50 completed runs across binary and multiclass tasks. Each dataset is processed as a time-ordered stream of flows, with ten-flow windows for the sequence models and KAN-window, while KAN-static receives only the final flow in each window. The evaluation uses 47 retained features, chronological train/validation/test partitions, boundary gaps, and class-weighted cross-entropy. The results vary by dataset. Flattening history for KAN generally offers little benefit and reduces several reported scores, although the difference is small or absent on some BoT-IoT measures. Sequence models outperform KAN-window on several ToN-IoT and BigFlow-NIDS comparisons, but this pattern is not universal across datasets and metrics. On NF-UNSW-NB15-v3, KAN-static records the highest accuracy and macro F1 among the evaluated models, although its macro F1 remains modest. On NF-CSE-CIC-IDS2018-v3, all binary models achieve 0.4402 accuracy and zero attack recall under the chronological split. A sequential audit of NF-BoT-IoT-v3 also shows long attack-dominated traffic phases separated by relatively narrow transition regions. These findings highlight the importance of evaluation order and dataset structure when interpreting NIDS results.**

***Index Terms*—network intrusion detection, Kolmogorov-Arnold networks, recurrent neural networks, Transformer, NetFlow v3, inter-packet arrival time, chronological evaluation, temporal distribution shift.**

I. Introduction

Network intrusion detection systems (NIDS) are commonly evaluated with flow records represented as tabular features. In an operating network, however, flows arrive over time, and traffic characteristics can change as services, devices, and attack activity change. Earlier work has discussed both the limitations of benchmark datasets and the difficulty of evaluating intrusion detectors under conditions that resemble deployment [1]–[3]. The choice of feature representation and the way data are partitioned therefore matter when the aim is to assess performance on later traffic [4]–[6].

Liu et al. introduced Kolmogorov-Arnold networks (KANs) in their ICLR 2025 paper [7], drawing on the Kolmogorov-Arnold representation theorem [8]. In a conventional multilayer perceptron (MLP), fixed activation functions are applied at the nodes. KAN instead places learnable univariate functions on the connections between nodes. The original work reported promising results on function approximation and scientific tasks [7]. KAN-based approaches have since been explored for intrusion detection, including a convolutional KAN model for IoT traffic and a study of KANs for intrusion detection [9], [10]. Those studies do not, however, establish how a standard KAN compares with sequence models when each model is given an ordered history of network flows.

This comparison raises a practical question because network-flow records are not necessarily independent observations. Consecutive flows may belong to the same burst or reflect successive stages of an attack [6]. At the same time, the timing information used here is recorded as flow-level statistics: the eight retained inter-packet arrival time (IAT) fields summarize timing within each flow. The models do not receive raw packet-level IAT sequences. Instead, the temporal models process ten consecutive flow records, allowing the experiment to test whether order across flows adds useful information [6], [11].

Random partitioning can place temporally adjacent flows, or records from the same attack period, in both training and test sets. This may make a test set less representative of the future traffic a deployed detector would encounter. Work on machine learning for computer security has documented how experimental design and dependence between samples can lead to overly optimistic estimates [2], [3], [12], [13].

Reshaping rows into a sequence does not by itself make the rows a meaningful time series. If row order is arbitrary or has been shuffled, the resulting input may not preserve the transitions that a recurrent or attention-based model is intended to learn. Sequence construction should therefore follow the event order recorded in the telemetry [11].

The NetFlow v2 collection provides 43 enhanced features, while v3 adds ten temporal fields, bringing the total to 53 [4],

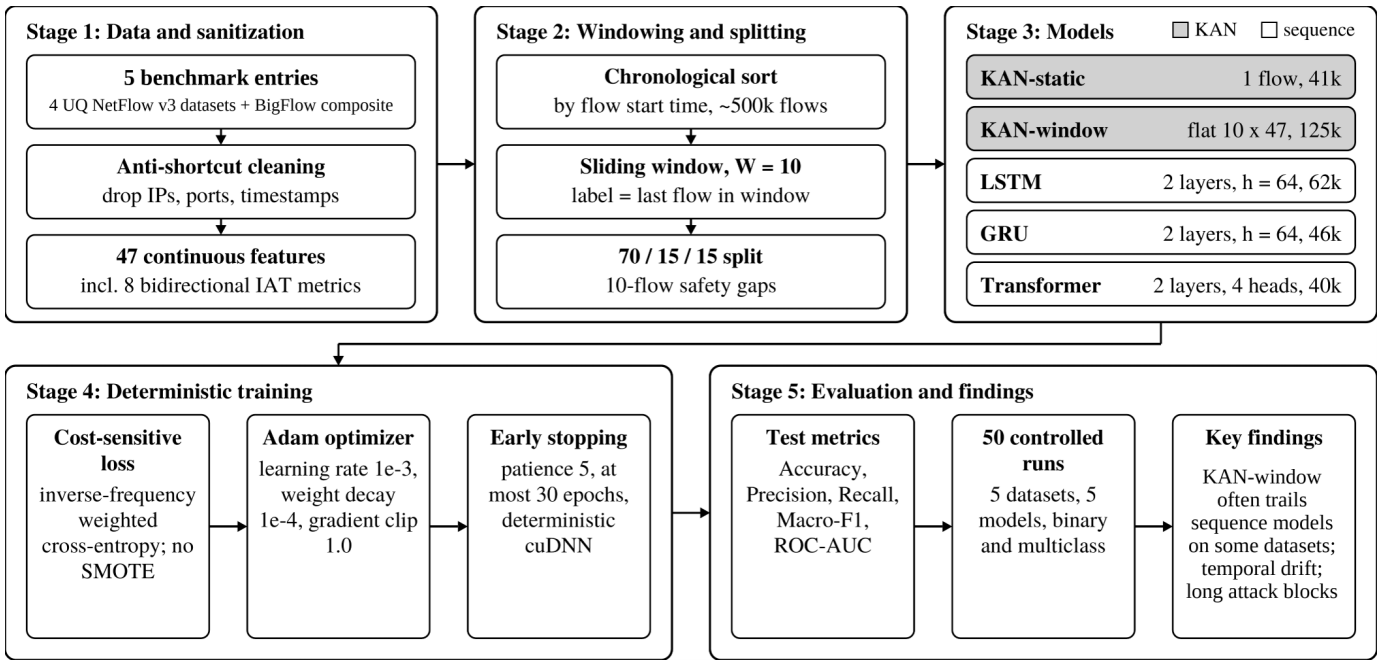


Fig. 1. Overview of the benchmark pipeline across the four University of Queensland NetFlow v3 datasets and the composite BigFlow-NIDS benchmark. The stages cover feature sanitization, chronological window construction, temporal train/validation/test partitions with boundary gaps, model training, and evaluation.

[6]. These additions include flow timestamps and bidirectional IAT statistics. In this study, timestamps are used to order the records and are then removed from the model inputs; eight IAT statistics and 39 other numerical fields remain.

This paper reports a controlled comparison across five benchmark entries: the four University of Queensland NetFlow v3 datasets (NF-ToN-IoT-v3, NF-BoT-IoT-v3, NF-UNSW-NB15-v3, and NF-CSE-CIC-IDS2018-v3) and BigFlow-NIDS, a composite collection built from those four sources [6], [14]–[21]. For each entry, flows are sorted by start time and evaluated in chronological partitions using ten-flow sliding windows and gaps between partitions. We compare KAN-static, which receives only the final flow, with KAN-window, which receives the flattened ten-flow window, and with LSTM, GRU, and Transformer models that process the window as a sequence. This design separates the effect of access to historical records from the effect of a sequence-based architecture.

II. System Methodology and Telemetry Formulation

A. NetFlow v3 Feature Telemetry Schema

The experiments use the four NetFlow v3 datasets released by the University of Queensland [21], together with BigFlow-NIDS, a composite release built from those datasets [19], [20]. The underlying source datasets are ToN-IoT, BoT-IoT, UNSW-NB15, and CSE-CIC-IDS2018 [14]–[18]. The University of Queensland describes its NetFlow v3 collection as extending the 43-feature v2 schema with ten temporal fields, for 53 flow features in total [6], [21]. The BigFlow-NIDS version-2 release lists 55 columns: 53 flow-feature fields plus two label fields, `Label` and `Attack` [19], [20]. In this study, the input pipeline excludes four host/port identifiers and two timestamps, leaving 47 numerical flow features. The 55-column count in the BigFlow release should therefore not be interpreted as 55 model input features.

Timing attributes (8 metrics): bidirectional minimum, maximum, average, and standard deviation of inter-packet arrival times: `SRC_TO_DST_IAT_{MIN, MAX, AVG, STDDEV}` and `DST_TO_SRC_IAT_{MIN, MAX, AVG, STDDEV}`.

Volumetric and directional attributes (39 metrics): packet counters, byte counters, TCP control flags (SYN, ACK, FIN, RST, PSH, URG), Internet Control Message Protocol (ICMP) types, Internet Protocol (IP) header fields, time-to-live (TTL) dynamics, and packet size histograms.

B. Anti-Shortcut Sanitization

The preprocessing removes fields that could let a classifier rely on host or port identity rather than traffic behaviour. This is a design choice for the present benchmark; it does not establish that every possible source of dataset-specific information has been removed.

Host and port scrubbing: `IPV4_SRC_ADDR`, `IPV4_DST_ADDR`, `L4_SRC_PORT`, and `L4_DST_PORT` are removed from the input features. This reduces the opportunity for the models to memorize address or port values associated with particular hosts or capture environments.

Timestamp handling: `FLOW_START_MILLISECONDS` and `FLOW_END_MILLISECONDS` are retained only for chronological sorting and window alignment. Neither timestamp is passed to the models.

Numerical preprocessing: 47 numerical flow features are retained. Nonnumeric and infinite values are replaced with zero, and extreme values are clipped to the representable range used for 32-bit floating-point inputs.

C. Chronological Sliding Windows and Boundary-Safe Splitting

Flows are ordered by `FLOW_START_MILLISECONDS`. A contiguous block of approximately 500,000 flows is selected from each dataset for the benchmark.

Each window contains ten consecutive flow records in chronological order: $X_i = [x_{t-9}, x_{t-8}, \ldots, x_t] \in \mathbb{R}^{10\times 47}$. The window therefore contains the current flow and the nine immediately preceding flows.

Training windows use a stride of five, producing approximately 70,000 windows. Validation and test windows use a stride of one, producing approximately 75,000 windows in each partition. The denser stride in the latter partitions allows the evaluation to cover each available transition in those contiguous blocks.

The target for each window is the label of its final flow, $y_i = \mathrm{label}(x_t)$. The preceding nine records provide context but do not determine the target.

A buffer of ten flows is omitted between the training and validation partitions and again between validation and test. This prevents any constructed window from crossing a partition boundary. It limits direct overlap between windows across the splits, but should not be read as proof that all traffic observations are statistically independent.

The scaler is fitted using the training partition only. SMOTE is not used because synthetic rows would not preserve the observed order of flows or the transitions represented by the windows [11], [22]. Instead, class imbalance is handled with inverse-frequency class-weighted cross-entropy. The weights are computed from the training-window counts [23]. We use the simple inverse-frequency weighting rule shown below, rather than the effective-number formulation proposed in [24]:

$$w_c = \frac{N}{C \cdot N_c} \tag{1}$$

where N is the total number of training windows, C is the number of classes, and N_c is the number of training windows of class c.

D. Model Architectures and Optimization

In a KAN layer, each connection between input coordinate j and output node i represents an adaptive univariate function $\phi_{i,j}(x_j)$ [7]. The pre-activation for output node i is:

$$y_i = \sum_j \phi_{i,j}(x_j) \tag{2}$$

Each activation is parameterized as:

$$\phi_{i,j}(x) = w_b \,\mathrm{SiLU}(x) + \sum_m c_{i,j,m} B_m(x) \tag{3}$$

Here, j indexes the input coordinates and m indexes the spline basis functions; d_{in} is the input dimension, while w_b and $c_{i,j,m}$ are learnable parameters. SiLU is the sigmoid-weighted linear unit [25], and B_m denotes a Cox–de Boor B-spline of degree $k = 2$ [26]. The implementation uses a spline grid with $G = 5$ intervals. The five evaluated architectures are described below:

KAN-static: receives only the current flow x_t (47 inputs). It uses quadratic B-splines over five grid intervals and LayerNorm; the model has 41,152 parameters in binary classification and 43,200 in multiclass classification.

KAN-window: receives the ten-flow window flattened into a 470-dimensional vector. It has 124,768 parameters in binary classification and 125,792 in multiclass classification.

LSTM: a two-layer stacked LSTM with hidden dimension 64 and dropout 0.3. The final hidden state is used for classification. The model has 62,338 binary and 62,858 multiclass parameters [27], [28].

GRU: a two-layer stacked GRU with hidden dimension 64 and dropout 0.3. Classification uses the final hidden state. The model has 46,786 binary and 47,306 multiclass parameters [29], [30].

Transformer encoder: a two-layer encoder with $d_{\mathrm{model}} = 48$, four attention heads, $d_{\mathrm{ff}} = 96$, and dropout 0.2. Sinusoidal positional encodings are added to the input sequence, and mean pooling is applied before classification. The model has 40,418 binary and 40,810 multiclass parameters [31], [32].

All experiments were run on an NVIDIA GeForce RTX 4060 Laptop GPU using PyTorch 2.6.0+cu124 and CUDA 12.4 [33]. The cuDNN configuration was set for deterministic execution [34]. Models were optimized with Adam [35], using an initial learning rate of 1e-3 and weight decay of 1e-4, together with a ReduceLROnPlateau scheduler. Gradient norms were clipped at 1.0 [36]. Training ran for at most 30 epochs, with early stopping after five epochs without validation improvement [37]. Accuracy, precision, recall, and macro F1 were calculated on the held-out chronological test partitions [38]. ROC-AUC was also reported using the standard ROC analysis framework [39].

III. Experimental Results and Empirical Analysis

The benchmark comprises 50 completed runs: five benchmark entries, five architectures, and two evaluation modes (binary and multiclass). All numerical results in Tables I–III are from experiments conducted by the authors. Table I provides model-level binary results for all five entries. Table II provides model-level multiclass results; for NF-CSE-CIC-IDS2018-v3, all five architectures produced identical metrics, shown on separate rows.

A. Does Giving KAN History Help It?

The results do not show a uniform benefit from giving KAN a longer input. On NF-ToN-IoT-v3, KAN-window has lower binary accuracy than KAN-static (0.9022 versus 0.9283) and lower multiclass macro F1 (0.9011 versus 0.9275). On NF-UNSW-NB15-v3, multiclass macro F1 falls from 0.4253 to 0.2731, a relative decline of 35.8%. On BigFlow-NIDS, binary F1 falls from 0.6439 to 0.6148. The BoT-IoT results are different: KAN-static and KAN-window have the same binary F1 and the same reported multiclass macro F1. On NF-CSE-CIC-IDS2018-v3, all five models produced identical

TABLE I
MASTER BINARY CLASSIFICATION RESULTS ($W = 10$)

Dataset	Model	Params	Accuracy	Precision	Recall	F1-Score	ROC-AUC
ToN-IoT-v3	KAN-static	41k	0.9283	0.8953	0.9456	0.9198	0.9279
	Transformer	40k	0.9188	0.8847	0.9350	0.9092	0.9508
	LSTM	62k	0.9088	0.8730	0.9249	0.8982	0.9390
	GRU	46k	0.9084	0.8739	0.9224	0.8975	0.9443
	KAN-window	125k	0.9022	0.8675	0.9148	0.8905	0.9334
BoT-IoT-v3	KAN-static	41k	0.9988	0.9988	1.0000	0.9994	0.8300
	KAN-window	125k	0.9988	0.9988	1.0000	0.9994	0.8266
	LSTM	62k	0.9988	0.9988	1.0000	0.9994	0.8767
	Transformer	40k	0.9988	0.9988	1.0000	0.9994	0.8179
	GRU	46k	0.7956	0.9993	0.7959	0.8861	0.8195
UNSW-NB15-v3	KAN-static	41k	0.9998	0.9974	0.9996	0.9985	0.99995
	Transformer	40k	0.9997	0.9954	1.0000	0.9977	0.99995
	LSTM	62k	0.9997	0.9952	1.0000	0.9976	0.99996
	GRU	46k	0.9996	0.9950	0.9998	0.9974	0.99995
	KAN-window	125k	0.9994	0.9924	0.9982	0.9953	0.99996
CSE-CIC-IDS2018-v3	Transformer	40k	0.4402	0.0000	0.0000	0.0000	0.6641
	KAN-static	41k	0.4402	0.0000	0.0000	0.0000	0.5809
	GRU	46k	0.4402	0.0000	0.0000	0.0000	0.4103
	KAN-window	125k	0.4402	0.0000	0.0000	0.0000	0.2901
	LSTM	62k	0.4402	0.0000	0.0000	0.0000	0.2341
BigFlow-NIDS	GRU	46k	0.9525	0.5981	0.8346	0.6968	0.9530
	Transformer	40k	0.9443	0.5493	0.8295	0.6609	0.9511
	LSTM	62k	0.9468	0.5677	0.7881	0.6600	0.9318
	KAN-static	41k	0.9437	0.5497	0.7769	0.6439	0.8969
	KAN-window	125k	0.9453	0.5702	0.6671	0.6148	0.9117

TABLE II
MASTER MULTICLASS CLASSIFICATION RESULTS ($W = 10$)

Dataset	Model	Params	Accuracy	Macro F1	Weighted F1	Macro Prec	Macro Rec
ToN-IoT-v3	KAN-static	41k	0.9283	0.9275	0.9285	0.9258	0.9303
	Transformer	40k	0.9188	0.9179	0.9190	0.9162	0.9206
	LSTM	62k	0.9088	0.9078	0.9091	0.9062	0.9107
	GRU	46k	0.9084	0.9073	0.9086	0.9057	0.9100
	KAN-window	125k	0.9022	0.9011	0.9025	0.8996	0.9037
BoT-IoT-v3	KAN-static	41k	0.9988	0.4997	0.9982	0.4994	0.5000
	KAN-window	125k	0.9988	0.4997	0.9982	0.4994	0.5000
	LSTM	62k	0.9988	0.4997	0.9982	0.4994	0.5000
	Transformer	40k	0.9988	0.4997	0.9982	0.4994	0.5000
	GRU	46k	0.7956	0.4461	0.8850	0.5012	0.6676
UNSW-NB15-v3	KAN-static	43k	0.9655	0.4253	0.9679	0.4026	0.6048
	Transformer	41k	0.9548	0.3137	0.9590	0.3257	0.4429
	KAN-window	126k	0.9487	0.2731	0.9529	0.2994	0.3610
	GRU	47k	0.9461	0.2722	0.9519	0.2985	0.3916
	LSTM	63k	0.9454	0.2483	0.9496	0.2843	0.3485
CSE-CIC-IDS2018-v3	KAN-static	41k	0.4402	0.3056	0.2691	0.2201	0.5000
	Transformer	40k	0.4402	0.3056	0.2691	0.2201	0.5000
	LSTM	62k	0.4402	0.3056	0.2691	0.2201	0.5000
	GRU	47k	0.4402	0.3056	0.2691	0.2201	0.5000
	KAN-window	125k	0.4402	0.3056	0.2691	0.2201	0.5000
BigFlow-NIDS	GRU	46k	0.9525	0.8355	0.9561	0.7931	0.8976
	LSTM	62k	0.9468	0.8156	0.9508	0.7762	0.8730
	Transformer	40k	0.9443	0.8153	0.9494	0.7685	0.8909
	KAN-static	41k	0.9437	0.8067	0.9481	0.7668	0.8662
	KAN-window	125k	0.9453	0.7946	0.9473	0.7733	0.8193

Note: All five NF-CSE-CIC-IDS2018-v3 models yielded identical multiclass metrics: 0.4402 accuracy, 0.3056 macro F1, 0.2691 weighted F1, 0.2201 macro precision, and 0.5000 macro recall. Each model collapsed to the dominant benign-class prediction under severe chronological distribution shift.

multiclass metrics, so this split does not distinguish between the architectures. Thus, flattening history often failed to improve KAN in these experiments, but the effect was dataset- and metric-dependent. KAN-window also increases the input size

from 47 to 470 coordinates and the binary parameter count from 41,152 to 124,768. The results are consistent with a mismatch between a flat coordinate representation and ordered temporal input; the experiment does not isolate that mechanism as the sole cause.

B. Equal History Comparison: Sequence Models Versus KAN-Window

The comparison is more mixed than a blanket statement that sequence models win on every dataset. On BigFlow-NIDS, GRU records binary F1/AUC of 0.6968/0.9530 and Transformer records 0.6609/0.9511, both above KAN-window at 0.6148/0.9117. On NF-ToN-IoT-v3, all three sequence models have higher binary AUC than KAN-window, and their multiclass macro F1 scores also exceed KAN-window. On NF-UNSW-NB15-v3, Transformer is above KAN-window in multiclass macro F1 (0.3137 versus 0.2731), while GRU and LSTM are slightly lower (0.2722 and 0.2483). On BoT-IoT, most models tie on binary F1 and multiclass macro F1. The GRU has lower accuracy and F1 scores, but higher multiclass macro precision and recall. On NF-CSE-CIC-IDS2018-v3, all five models produced identical multiclass metrics, so there is no model-level ranking on that split. These results suggest that sequential processing can help on some datasets and metrics, but its benefit is not consistent across all five benchmarks.

C. The Standing of KAN-Static as an Instantaneous Classifier

KAN-static remains a useful per-flow reference point in this benchmark. On NF-UNSW-NB15-v3, it records the highest multiclass accuracy (0.9655) and macro F1 (0.4253) among the five evaluated models. The macro F1 is nevertheless much lower than the accuracy, which indicates that performance is uneven across classes and that the high overall accuracy should not be interpreted as uniformly strong detection of minority attacks. KAN-static is therefore best viewed here as a competitive instantaneous classifier, not as evidence that historical context is generally unnecessary.

IV. Critical Empirical Failure Modes

A. Temporal Distribution Drift on NF-CSE-CIC-IDS2018-v3

Under the chronological split, all five binary models on NF-CSE-CIC-IDS2018-v3 achieve the same accuracy (0.4402), with precision, recall, and F1 all equal to zero. The test partition contains 74,971 windows: 33,001 benign (44.02%) and 41,970 FTP-BruteForce (55.98%). Validation loss was lowest at epoch 1 and increased monotonically through epoch 6, when early stopping occurred. Using the best checkpoint, each model classified all test windows as benign, correctly classifying the 33,001 benign windows but detecting none of the attack windows. The observed outcome is consistent with a change in attack traffic patterns between the earlier training period and the later test period. No matched randomized-split control was run, so this experiment does not quantify how much a random split would inflate the score.

TABLE III
Authors' Own Binary F1 Results for Static KAN (v2) vs. Sequence Models (v3)

Dataset	Static KAN (v2)	Transformer (v3)	GRU (v3)	LSTM (v3)
NF-UNSW-NB15	0.9528	0.9977	0.9974	0.9976
NF-ToN-IoT	0.9912	0.9092	0.8975	0.8982
NF-BoT-IoT	0.9996	0.9994	0.8861	0.9994
CSE-CIC-2018	0.9656	0.0000[a]	0.0000[a]	0.0000[a]
BigFlow-NIDS	—	0.6609	0.6968	0.6600

[a]The NetFlow v3 models returned zero attack recall under the chronological NF-CSE-CIC-IDS2018-v3 test split. This table provides contextual comparison across benchmark versions and feature sets; it is not a matched architecture comparison.

B. Long, Nonrecurring Attack Blocks in NF-BoT-IoT-v3

The sequential audit covers 16,933,808 rows in the local NF-BoT-IoT-v3 file used for this study, divided into 34 contiguous chunks. The University of Queensland dataset portal currently lists a headline total of 16,993,808 flows, while the class counts displayed on that page sum to 16,933,844. Those public figures do not reconcile with each other or with the local file audited here. We therefore retain 16,933,808 as the exact row count of the file used in our experiment, without treating it as a correction to the portal's published total [21]. The audit identifies long periods dominated by DDoS, followed by a DoS-dominated period and then a reconnaissance-dominated period; theft appears only near the end of the capture. The chunk-level breakdown also shows mixed-label boundary chunks: the first chunk contains a small benign component, the transition into reconnaissance includes benign flows, and the final chunk contains reconnaissance, benign, and theft. The evidence therefore supports the presence of long, mostly nonrecurring attack blocks, but not the stronger claim that every contiguous 500,000-flow block is single-class or that multiclass chronological windows are impossible. The concentration of attack categories into long phases still limits the variety of class transitions available for chronological evaluation.

C. Cross-Paradigm Comparison of Static and Sequence Models

Table III also reports results from experiments conducted by the authors. It compares the authors' static KAN results on NetFlow v2 with their sequence-model results on NetFlow v3. These are original experimental results, not values reproduced from prior publications. The v2 and v3 versions use different feature sets [4], [6], [21], so the table is a contextual cross-version comparison rather than a controlled experiment that isolates architecture as the only variable.

For NF-UNSW-NB15, Table III lists binary F1 values of 0.9977, 0.9974, and 0.9976 for the v3 Transformer, GRU, and LSTM, respectively, compared with 0.9528 for the v2 static KAN entry. The current v3 benchmark also includes KAN-static, which records 0.9985 binary F1 in Table I. Because the v2 and v3 entries were not produced under a matched feature

and split protocol, these numbers do not establish that sequence architectures outperform KAN in general.

For NF-ToN-IoT, the v2 static KAN entry in Table III is 0.9912, while the v3 sequence-model values range from 0.8975 to 0.9092. This difference may reflect the change in dataset version, features, or evaluation protocol, so it should not be attributed to model architecture alone.

For NF-CSE-CIC-IDS2018, the table contrasts a v2 static KAN F1 of 0.9656 with zero F1 for the v3 sequence models under chronological evaluation. The v3 binary results in Table I show that all five tested architectures share the same zero-recall outcome. This contrast illustrates how strongly the evaluation conditions and temporal split can affect the reported result; it is not a controlled cross-version experiment.

V. Conclusion and Operational Guidelines

The results point to several practical conclusions, with the qualification that they are based on five selected datasets and a ten-flow window:

- **Use KAN-static as a per-flow baseline** when the task is to classify individual flow records without access to an ordered history. It performed well on several measures and achieved the highest accuracy and macro F1 among the evaluated models on NF-UNSW-NB15-v3, although the macro F1 shows that minority-class performance remains a concern.
- **Evaluate sequence models** when ordered flow windows are available. GRU and Transformer gave the strongest results on selected BigFlow-NIDS measures, but sequence models did not lead on every dataset or metric. Model selection should therefore be based on the target dataset and evaluation protocol.
- **Do not assume that adding history to KAN will improve its predictions.** The flattened KAN-window input often reduced performance compared with KAN-static and was generally less competitive than sequence processing on the datasets where sequence models showed gains. BoT-IoT also produced ties, so the effect is not identical across all datasets.
- **Use chronological partitions and boundary gaps** when the intended use is prediction on future traffic. Random partitions can mix related periods across training and test sets and may conceal temporal changes. The ten-flow gap used here prevents windows from crossing split boundaries; it should not be described as proof that all forms of leakage have been eliminated.

VI. Limitations

The window length was fixed at $W = 10$ to keep the 50-run benchmark within the available compute budget. Longer windows (for example, 50 or 100 flows) and session- or burst-level grouping were not evaluated, so the results may not extend to attacks that unfold over longer periods. The models were trained offline; the NF-CSE-CIC-IDS2018-v3 result shows that a model selected on earlier traffic may not transfer to a later period when traffic behaviour changes. Online updating and explicit drift detection were outside the scope of this experiment. The benchmark also uses selected contiguous portions of the source datasets, so its results should not be taken as a complete characterization of every traffic period in those collections. BigFlow-NIDS is itself a composite dataset built from NF-UNSW-NB15-v3, NF-ToN-IoT-v3, NF-BoT-IoT-v3, and NF-CSE-CIC-IDS2018-v3 [19], [20]. It therefore shares source datasets with four of the other benchmark entries and should not be treated as an independent fifth traffic source.

References

[1] M. Ring, S. Wunderlich, D. Scheuring, D. Landes, and A. Hotho, "A survey of network-based intrusion detection data sets," *Computers & Security*, vol. 86, pp. 147–167, 2019, doi: 10.1016/j.cose.2019.06.005.

[2] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symposium on Security and Privacy (S&P)*, 2010, pp. 305–316, doi: 10.1109/SP.2010.25.

[3] E. K. Viegas, A. O. Santin, and L. S. Oliveira, "Toward a reliable anomaly-based intrusion detection in real-world environments," *Computer Networks*, vol. 127, pp. 200–216, 2017, doi: 10.1016/j.comnet.2017.08.013.

[4] M. Sarhan, S. Layeghy, and M. Portmann, "Towards a standard feature set for network intrusion detection system datasets," *Mobile Networks and Applications*, vol. 27, no. 1, pp. 357–370, 2022, doi: 10.1007/s11036-021-01843-0.

[5] M. Sarhan, S. Layeghy, and M. Portmann, "Evaluating standard feature sets towards increased generalisability and explainability of ML-based network intrusion detection," *Big Data Research*, vol. 30, Art. no. 100359, 2022, doi: 10.1016/j.bdr.2022.100359.

[6] M. Luay, S. Layeghy, N. Noorbin, M. Sarhan, G. Kulatilleke, N. Moustafa, and M. Portmann, "Time matters: Temporal NetFlow features for ML-based network intrusion detection," *IEEE Access*, vol. 14, pp. 66899–66913, 2026, doi: 10.1109/ACCESS.2026.3688204.

[7] Z. Liu, Y. Wang, S. Vaidya, F. Ruehle, J. Halverson, M. Soljačić, T. Y. Hou, and M. Tegmark, "KAN: Kolmogorov-Arnold Networks," in *Proc. International Conference on Learning Representations (ICLR)*, 2025. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2025/hash/afaed89642ea100935e39d39a4da602c-Abstract-Conference.html.

[8] A. N. Kolmogorov, "On the representation of continuous functions of many variables by superposition of continuous functions of one variable and addition," *Dokl. Akad. Nauk SSSR*, vol. 114, pp. 953–956, 1957.

[9] M. Abd Elaziz, I. Ahmed Fares, and A. O. Aseeri, "CKAN: Convolutional Kolmogorov-Arnold networks model for intrusion detection in IoT environment," *IEEE Access*, vol. 12, pp. 134837–134851, 2024, doi: 10.1109/ACCESS.2024.3462297.

[10] P. González Santamarta, "Kolmogorov-Arnold networks for the development of intrusion detection systems," in *Advances in Computational Intelligence (IWANN 2025)*, Lecture Notes in Computer Science, vol. 16009, pp. 44–56. Cham, Switzerland: Springer, 2026, doi: 10.1007/978-3-032-02728-3_4.

[11] T. G. Dietterich, "Machine learning for sequential data: A review," in *Structural, Syntactic, and Statistical Pattern Recognition*, T. Caelli, Ed., Lecture Notes in Computer Science, vol. 2396. Berlin, Germany: Springer, 2002, pp. 15–30.

[12] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro, "TESSERACT: Eliminating experimental bias in malware classification across space and time," in *Proc. USENIX Security Symposium*, 2019, pp. 729–746.

[13] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *Proc. 31st USENIX Security Symposium*, 2022, pp. 3971–3988.

[14] A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood, and A. Anwar, "TON_IoT telemetry dataset: A new generation dataset of IoT and IIoT for data-

driven intrusion detection systems," *IEEE Access*, vol. 8, pp. 165130–165150, 2020, doi: 10.1109/ACCESS.2020.3022862.

[15] N. Koroniotis, N. Moustafa, E. Sitnikova, and B. Turnbull, "Towards the development of realistic botnet dataset in the Internet of Things for network forensic analytics: Bot-IoT dataset," *Future Generation Computer Systems*, vol. 100, pp. 779–796, 2019, doi: 10.1016/j.future.2019.05.041.

[16] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems," in *Proc. IEEE Military Communications and Information Systems Conference (MilCIS)*, 2015, pp. 1–6, doi: 10.1109/MilCIS.2015.7348942.

[17] Canadian Institute for Cybersecurity, University of New Brunswick, "CSE-CIC-IDS2018 on AWS." [Online]. Available: https://www.unb.ca/cic/datasets/ids-2018.html. Accessed: Nov. 2, 2026.

[18] AWS Registry of Open Data, "A Realistic Cyber Defense Dataset (CSE-CIC-IDS2018)." [Online]. Available: https://registry.opendata.aws/cse-cic-ids2018/. Accessed: Nov. 2, 2026.

[19] M. B. Uddin, M. S. Arefin, and M. M. M. Hussain, "BigFlow-NIDS: A large-scale dataset for network intrusion detection in big data environment," *Data in Brief*, vol. 65, Art. no. 112530, Apr. 2026, doi: 10.1016/j.dib.2026.112530.

[20] M. B. Uddin and M. S. Arefin, "BigFlow-NIDS," Mendeley Data, ver. 2, Jan. 22, 2026, doi: 10.17632/nv729tbdgz.2.

[21] University of Queensland, "Machine Learning-Based NIDS Datasets." [Online]. Available: https://staff.itee.uq.edu.au/marius/NIDS_datasets/. Accessed: Nov. 2, 2026.

[22] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002, doi: 10.1613/jair.953.

[23] J. M. Johnson and T. M. Khoshgoftaar, "Survey on deep learning with class imbalance," *Journal of Big Data*, vol. 6, Art. no. 27, 2019, doi: 10.1186/s40537-019-0192-5.

[24] Y. Cui, M. Jia, T.-Y. Lin, Y. Song, and S. Belongie, "Class-balanced loss based on effective number of samples," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 9268–9277, doi: 10.1109/CVPR.2019.00949.

[25] S. Elfwing, E. Uchibe, and K. Doya, "Sigmoid-weighted linear units for neural network function approximation in reinforcement learning," *Neural Networks*, vol. 107, pp. 3–11, 2018.

[26] C. de Boor, *A Practical Guide to Splines*, Applied Mathematical Sciences, vol. 27. New York, NY, USA: Springer-Verlag, 1978.

[27] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[28] F. A. Gers, J. Schmidhuber, and F. Cummins, "Learning to forget: Continual prediction with LSTM," *Neural Computation*, vol. 12, no. 10, pp. 2451–2471, 2000.

[29] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proc. EMNLP*, 2014, pp. 1724–1734, doi: 10.3115/v1/D14-1179.

[30] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," *arXiv preprint arXiv:1412.3555*, 2014.

[31] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, pp. 5998–6008, 2017.

[32] Z. Wu, H. Zhang, P. Wang, and Z. Sun, "RTIDS: A robust Transformer-based approach for intrusion detection system," *IEEE Access*, vol. 10, pp. 64375–64387, 2022, doi: 10.1109/ACCESS.2022.3182333.

[33] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, vol. 32, pp. 8024–8035, 2019.

[34] PyTorch Contributors, "Reproducibility," PyTorch documentation, version 2.6.0. [Online]. Available: https://docs.pytorch.org/docs/2.6/notes/randomness.html. Accessed: Nov. 2, 2026.

[35] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. International Conference on Learning Representations (ICLR)*, 2015, doi: 10.48550/arXiv.1412.6980.

[36] R. Pascanu, T. Mikolov, and Y. Bengio, "On the difficulty of training recurrent neural networks," in *Proc. International Conference on Machine Learning (ICML)*, 2013, pp. 1310–1318.

[37] L. Prechelt, "Automatic early stopping using cross validation: Quantifying the criteria," *Neural Networks*, vol. 11, no. 4, pp. 761–767, Jun. 1998, doi: 10.1016/S0893-6080(98)00010-0.

[38] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," *Information Processing & Management*, vol. 45, no. 4, pp. 427–437, 2009, doi: 10.1016/j.ipm.2009.03.002.

[39] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006, doi: 10.1016/j.patrec.2005.10.010.